Journal of
Information Systems Engineering
and Business Intelligence

Vol.X, No.X, February/June/October XXXX
Available online at: http://e-journal.unair.ac.id/index.php/JISEBI

SQL Injection Risk Analysis in Web-Based Applications Using the Penetration Testing Method and CVSS v3.1

Reinaldy Thendean [1)], Fenina Adline Twince Tobing [2)*]

[1)2)]*Informatics Study Program, Faculty of Engineering and Informatics, Universitas Multimedia Nusantara, Tangerang, Indonesia*

[1)]reinaldy.thendean@student.umn.ac.id, [2)*]fenina.tobing@umn.ac.id

Abstract

Background: SQL injection (SQLi) remains one of the most prevalent threats to web application security and continues to be classified as a significant risk in the OWASP Top 10 (2025). Prior studies frequently demonstrate SQLi exploitation without integrating the results with a systematic, measurable risk evaluation, leaving a gap between practical exploitation and standardized severity assessment.

Objective: This study aims to identify and exploit SQLi vulnerabilities in a web application using a penetration testing approach and to analyze their severity using the Common Vulnerability Scoring System (CVSS) v3.1, thereby integrating exploitation evidence with quantifiable, per-parameter risk scoring.

Methods: A limited black-box penetration test was conducted in a controlled, intentionally vulnerable Docker-based laboratory environment. Manual testing was performed first to detect indications of the vulnerability, followed by automated exploitation using SQLMap version 1.9.4. Each CVSS v3.1 base metric was then assessed against the empirical exploitation evidence, and the resulting score was cross-validated against prior literature.

Results: The login feature was vulnerable to authentication bypass, while the search feature allowed SQLMap to extract all 15 user credential records, including passwords stored in plaintext, using time-based blind and UNION query techniques. The analysis produced the vector CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N with a Base Score of 7.5 (High), consistent with prior studies and verified by manual calculation.

Conclusion: Integrating practical exploitation with per-parameter CVSS assessment yields a transparent and reproducible severity evaluation. The study recommends prepared statements, server-side input validation, and password hashing as primary mitigations.

Keywords: CVSS v3.1, Penetration Testing, SQL Injection, SQLMap, Web Application Security

Article history: Received __ ____ 20XX, first decision __ ____ 20XX, accepted __ ____ 20XX, available online __ ____ 20XX

I. INTRODUCTION

In the current digital era, web applications have become an essential part of modern information technology infrastructure. Critical services such as e-commerce, online banking, and hospital management systems increasingly rely on web-based platforms [1], [2]. This dependence widens the attack surface: the Cybersecurity and Infrastructure Security Agency (CISA) reported that more than 70% of organizations worldwide experienced at least one cybersecurity incident during 2023 [2].

Among the attacks that persist against web applications, SQL injection remains one of the most common, with an estimated 340 million attempts per year [4]. SQL injection is a vulnerability that allows an attacker to manipulate database commands through improperly validated input [5]. Although the vulnerability has long been known, it remains widespread: according to the OWASP Top 10 (2025) report [6], the Injection category still constitutes a significant class of web application security weaknesses, with a maximum incidence rate of 13.77% and an average of 3.08% across a large corpus of tested applications. Through this technique, an attacker may access or modify data without authorization, compromise data integrity, or gain administrative access to the database [7].

The impact of SQL injection can be severe. A well-known example is the attack on a national job portal in India that exposed the personal data of approximately 30 million job seekers [5]. Recent research also shows that SQL injection vulnerabilities can reach CVSS scores as high as 9.1 (Critical), reflecting the potentially serious consequences for the confidentiality, integrity, and availability of data [4].

Penetration testing is a widely used approach for identifying and evaluating vulnerabilities before they are exploited by malicious parties [3], and tools such as SQLMap can automate the identification and exploitation of SQL injection flaws [7]. The Common Vulnerability Scoring System (CVSS) is commonly used as a standard for rating severity [8]. However, existing analyses often do not examine individual CVSS parameters in depth [4], nor do they systematically integrate exploitation results with severity assessment [5]. Many SQL injection studies focus on identifying or demonstrating exploitation without an accompanying measurable risk evaluation [3], [9], so work that integrates the exploitation process with CVSS-based severity analysis remains relatively limited.

ISSN 2443-2555 (online) 2598-6333 (print) © 20XX The Authors. Published by Universitas Airlangga.
This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/)
doi: http://dx.doi.org/10.20473/jisebi.X.X.pp-pp

To address this gap, this study performs both exploitation of SQL injection vulnerabilities and a systematic analysis of their severity using a measurable approach. The contributions are twofold: (1) it identifies and exploits SQL injection vulnerabilities in a web application using penetration testing and analyzes their impact on data security; and (2) it analyzes the severity and risk of the identified vulnerabilities using CVSS v3.1, providing an explicit per-parameter justification that links each metric value to the empirical exploitation evidence.

II. Literature Review

A. SQL Injection

SQL injection (SQLi) is a security vulnerability in web applications that enables an attacker to insert malicious SQL commands into database queries [10]. It occurs when user input is not properly validated and is used directly in an SQL query, allowing the attacker to alter the structure of the executed query [11]. For example, an authentication query of the form SELECT * FROM users WHERE username = 'admin' AND password = '123' can be manipulated into a condition that is always true, thereby bypassing authentication. The root causes include the absence of input validation and sanitization [10], [5], dynamic string building without parameterization, the use of database accounts with excessive privileges, and error handling that exposes sensitive information [11], [10].

Several SQLi techniques exist. Tautology-based injection exploits an always-true logical condition (e.g., ' OR 1=1 --) to bypass authentication [5], [10]. Union-based injection uses the UNION operator to append the results of a malicious query to those of the original query, enabling direct data extraction [5]. Error-based injection leverages database error messages to reveal internal information [10]. Blind injection is used when the application returns neither output nor error messages: boolean-based blind relies on differences in application responses to true and false conditions [11], while time-based blind uses delay functions such as SLEEP() to infer information from response timing [10].

B. Penetration Testing and SQLMap

Penetration testing is a systematic process for evaluating the security of a system by simulating controlled attacks to identify vulnerabilities [12], [13]. It is conducted legally and ethically, with authorization from the system owner [14]. The methodology typically comprises five phases: reconnaissance, scanning, gaining access (exploitation), maintaining access, and covering tracks [12], [14]. Testing approaches are classified as white-box, black-box, or gray-box depending on the level of prior knowledge of the target [13], [14].

SQLMap is an open-source penetration testing tool designed specifically to detect and exploit SQL injection vulnerabilities in web applications [17], [18]. It supports multiple techniques, including boolean-based blind, error-based, union-based, and time-based blind injection, and can automatically enumerate databases, tables, and columns and extract data [17], [19]. Prior studies confirm that SQLMap can automate the extraction of authentication data such as usernames and passwords [18], [20], although its coverage may not fully reflect real-world conditions [18], [21].

C. Common Vulnerability Scoring System (CVSS v3.1)

CVSS is an open framework for describing and measuring the severity of vulnerabilities, producing a numeric score (0.0 to 10.0) and a textual vector string [8]. It comprises three metric groups: Base, Temporal, and Environmental. The Base Metrics represent the intrinsic characteristics of a vulnerability that do not depend on time or environment and are used to compute the Base Score. They consist of exploitability sub-metrics (Attack Vector, Attack Complexity, Privileges Required, User Interaction) and impact sub-metrics (Confidentiality, Integrity, Availability), together with Scope. Scores are mapped onto a qualitative rating scale: None (0.0), Low (0.1 to 3.9), Medium (4.0 to 6.9), High (7.0 to 8.9), and Critical (9.0 to 10.0) [8].

D. Related Work

Several studies have applied CVSS v3.1 to rate web application vulnerabilities. Utama et al. identified an SQL injection vulnerability on the login page of an academic information system with a CVSS score of 7.4 (High), explicitly assigning a high confidentiality impact because exploitation granted full access to sensitive data [22]. Febrian et al. found an SQL injection vulnerability on a university website with a CVSS score of 9.0 (Critical), as well as an Insecure Direct Object Reference (IDOR) vulnerability scored 7.5 (High) [23]. Rohmaniah et al. reported a Directory Listing vulnerability categorized as High, exposing user credentials stored in an unprotected directory [24]. Complementarily, Neupane proposed a three-phase pipeline (preparation, execution, patching) but did not use a standardized risk-scoring method such as CVSS [5], while Adrian et al. integrated CVSS, OWASP, and APPI for SQLi risk analysis but did not detail the assessment of each CVSS parameter [4]. Beyond security-focused studies,

web-based information systems for administrative tasks such as tuition-fee payment and employee-leave management routinely store personal and financial records, which underscores the importance of protecting them against attacks such as SQL injection [25], [26]. Consistent with the present work, other penetration-testing studies have confirmed SQL injection as a high-severity vulnerability in web applications and recommended input validation, parameterized queries, and countermeasure assessment as primary mitigations [27]–[30]. These works motivate an approach that both exploits the vulnerability and transparently justifies each CVSS parameter.

III. METHODS

This study employed a limited penetration testing method with a black-box testing approach, chosen because it simulates real-world attack conditions performed by an external party without access to the source code or internal information of the target system. The overall procedure proceeds from target and parameter identification, through initial manual testing and automated exploitation with SQLMap, to CVSS v3.1 severity analysis and literature validation, as summarized in Fig. 1.

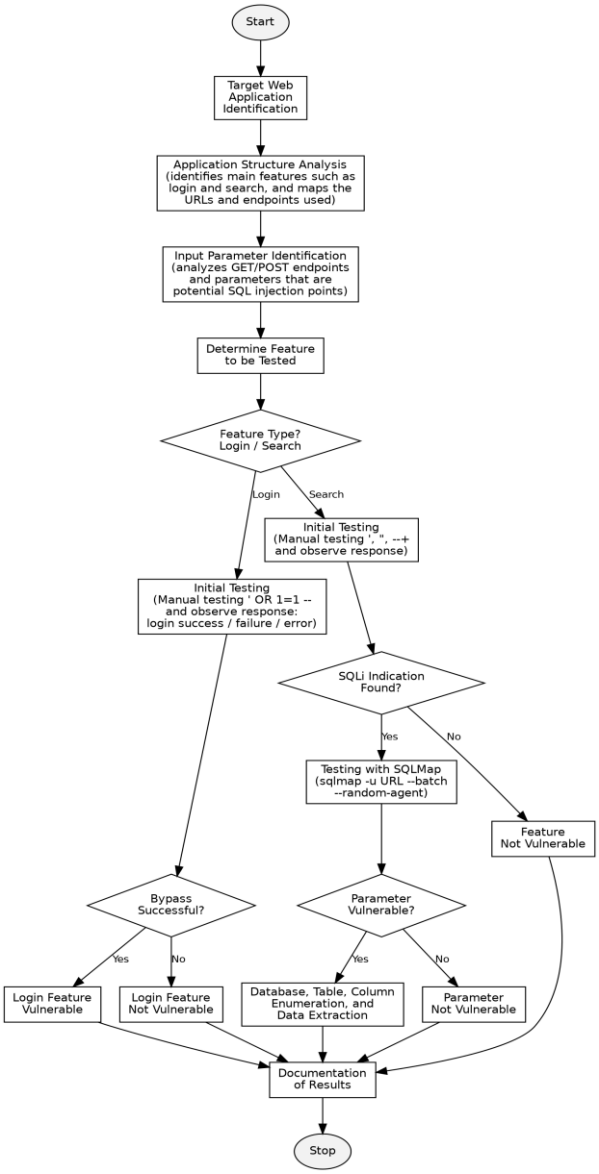


Fig. 1 Penetration testing procedure, from target identification to documentation of exploitation results.

A. Research Environment

The testing environment was a deliberately vulnerable, database-backed web application deployed locally with Docker on a Kali Linux 2024.2 virtual machine. The application was instantiated from the Docker Hub image zxxsnxx/vulnlabvuzlar and served on a dedicated port, enabling isolated, safe, and reproducible experimentation without affecting external systems. The back-end database was MySQL (MariaDB fork). The host used Windows 11 with an AMD Ryzen 9 6900HX processor and 32 GB of RAM, and the application was accessed through Mozilla Firefox. Exploitation was carried out with SQLMap version 1.9.4.

The use of an intentionally vulnerable application is an established and accepted practice in cybersecurity research [15], [16]. It provides an isolated environment in which exploitation can be performed thoroughly without legal risk or damage to production systems. The selected object of study exposes a login feature (POST method with authentication) and a search feature (GET method) that interact directly with the database through unsanitized queries, which are the two input-to-database interaction points most commonly found in real web applications and the most frequently exploited SQL injection attack surface.

B. Penetration Testing Procedure

The penetration testing was conducted in a structured, staged manner. First, the target and its input parameters were identified during a reconnaissance phase, cataloguing GET and POST entry points. Testing then focused on the login and search features because they interact directly with the database and are accessible without prior authentication, consistent with a black-box test performed with ordinary user privileges.

Initial manual testing was performed to detect early indications of the vulnerability by inserting special characters such as a single quote (') and simple logic-based payloads such as ' OR 1=1 --, while observing the application's responses (database errors, blank pages, or unexpected content changes). If an indication was found, the parameter was escalated to automated testing with SQLMap using its default configuration (sqlmap -u <URL> --dbs --batch --random-agent), progressing through database, table, and column enumeration to data extraction as proof of concept.

C. Severity Analysis and Validation

The severity of the confirmed vulnerability was assessed using the CVSS v3.1 Base Metrics. Each parameter was assigned a value objectively, based on the actual conditions observed during exploitation, and documented together with its justification. The Base Score was computed using the official FIRST CVSS v3.1 calculator and verified by manual calculation. The resulting score and severity category were then validated against prior literature published within the last five years (2020 to 2025) that addresses SQL injection or similar web application vulnerabilities and reports CVSS scores or vulnerability impact explicitly, focusing on the consistency of the severity category with the real-world impact reported in the literature rather than on obtaining an identical numeric score.

IV. RESULTS

A. Entry Point Identification

Two primary user-input entry points were identified. The login form transmits the email and password fields via the POST method for authentication. The search feature transmits its input via the GET method, with the parameter visible directly in the URL (e.g., ?search=tes). Because both interact directly with server-side database queries, they became the main targets for testing.

B. Initial Manual Testing

On the login feature, inserting a single quote (') into the email field caused the application to return an SQL syntax error, indicating that user input was passed directly into the query without validation. A ' OR 1=1 -- payload placed only in the email field did not bypass authentication, suggesting the email and password fields were validated separately. When the same payload was placed in both fields simultaneously, the system granted access without valid credentials, confirming an authentication bypass vulnerability. Because this outcome already demonstrated the bypass, the login feature was not escalated to SQLMap.

On the search feature, searching for a valid username (angelo) returned the corresponding record normally. Modifying the input to angelo'-- produced a blank page, because the closing quote of the original query was commented out and the WHERE clause returned no matching rows, which is the first strong indication of injectability. Refining the payload to angelo'--+ (where + is a URL-encoded space that satisfies MySQL's requirement for a whitespace character after the -- comment token) restored the normal result, as shown in Fig. 2. The contrast between

these two responses confirmed that the injection point was fully controllable, and the feature was escalated to SQLMap.

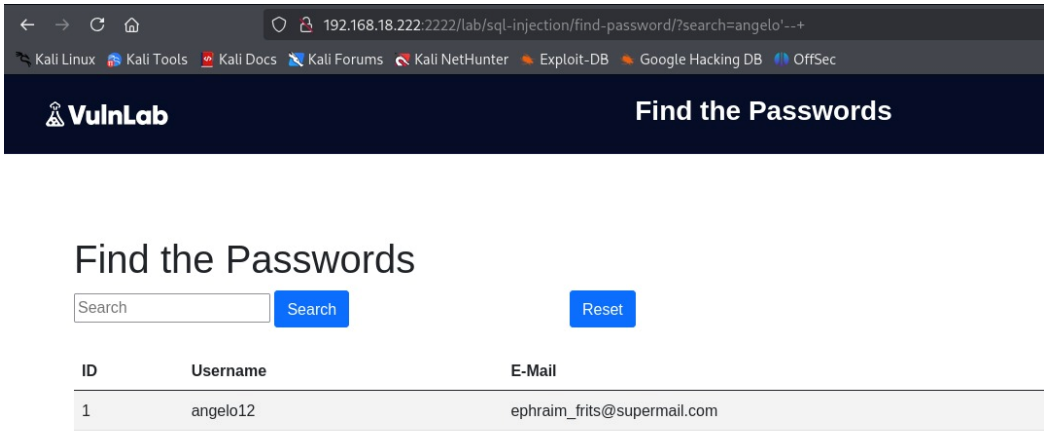


Fig. 2 Search-feature response to the angelo'--+ payload, confirming a controllable injection point.

C. Automated Exploitation with SQLMap

SQLMap confirmed that the search parameter was vulnerable to two techniques: time-based blind injection (using the MySQL SLEEP function) and UNION query injection (which SQLMap determined required six columns via automated ORDER BY probing). Enumeration proceeded systematically. Four databases were identified (information_schema, mysql, performance_schema, and sql_injection), of which sql_injection is the application database. Within it, three tables were found (images, stocks, and users), and the users table contained six columns: id, email, name, password, surname, and username.

Finally, the dump operation extracted all 15 user records in full, including credentials (Fig. 3). Critically, the passwords were stored in plaintext, so the extracted data could be used directly without any additional decryption step. A summary of the manual and automated testing is presented in Table 1.

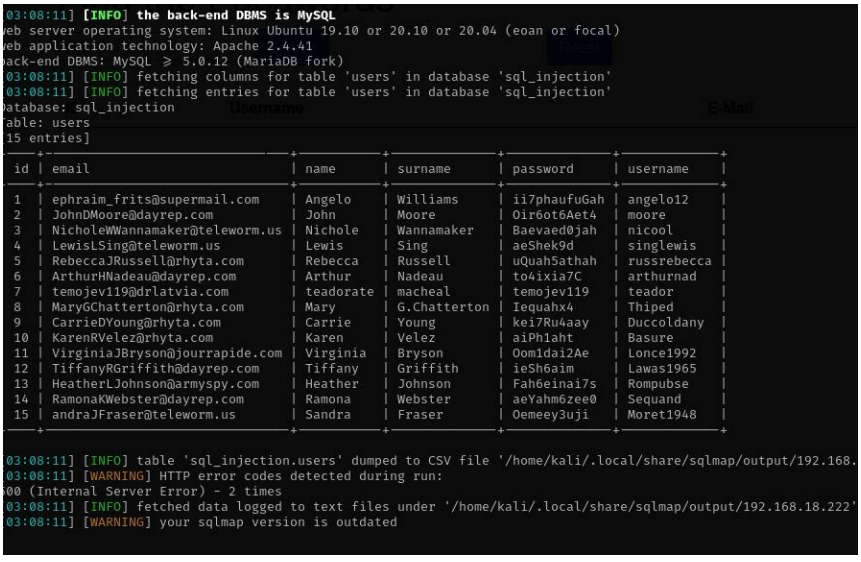


Fig. 3 SQLMap extraction of the users table, revealing 15 credential records including plaintext passwords.

TABLE 1
Summary of SQL injection testing on the vulnerable web application

Phase	Feature	Payload / Command	Result
Manual	Login	'	Database SQL syntax error, indicating unsanitized input.
Manual	Login	' OR 1=1 -- (email + password)	Authentication bypassed without valid credentials.
Manual	Search	angelo'--	Blank page; query structure broken (comment not yet valid).
Manual	Search	angelo'--+	Normal data returned; injection point fully controllable.
Automated	Search	sqlmap --dbs	Time-based blind and UNION (6 columns) confirmed; 4 databases enumerated.
Automated	Search	sqlmap -D sql_injection --tables	3 tables enumerated: images, stocks, users.
Automated	Search	sqlmap -T users --columns	6 columns: id, email, name, password, surname, username.
Automated	Search	sqlmap -T users --dump	All 15 records extracted, including plaintext passwords.

D. CVSS v3.1 Severity Assessment

The CVSS analysis was focused on the search feature because its impact could be quantified directly through the extraction of 15 complete credential records, whereas the login bypass produced a conditional impact dependent on the account being bypassed. The assessment of each base metric, with its justification, is presented in Table 2.

TABLE 2
CVSS v3.1 base metric assessment

Parameter	Value	Justification
Attack Vector (AV)	Network (N)	Exploitable remotely over the network by sending a crafted payload through the search parameter; no physical or local access required.
Attack Complexity (AC)	Low (L)	Deterministic and repeatable with consistent success, requiring no special conditions such as race conditions or specific timing.
Privileges Required (PR)	None (N)	The search feature is publicly accessible without prior authentication.
User Interaction (UI)	None (N)	Performed solely at the attacker's discretion; no action by another user is required.
Scope (S)	Unchanged (U)	Impact confined to the application database; it does not extend to the operating system or other components.
Confidentiality (C)	High (H)	All 15 credential records were extracted in full, including plaintext passwords, resulting in total loss of confidentiality.
Integrity (I)	None (N)	Exploitation was read-only (dump); no actual data modification was demonstrated.
Availability (A)	None (N)	No service disruption, resource exhaustion, or performance degradation was observed.

The Base Score was verified by manual calculation using the CVSS v3.1 equations. The exploitability sub-score is 8.22 x AV x AC x PR x UI = 8.22 x 0.85 x 0.77 x 0.85 x 0.85 = 3.877. The impact sub-score component ISC_Base = 1 - (1 - C)(1 - I)(1 - A) = 1 - (1 - 0.56)(1 - 0)(1 - 0) = 0.56, and, with Scope Unchanged, Impact = 6.42 x 0.56 = 3.595. Therefore Base Score = RoundUp(min(3.595 + 3.877, 10)) = RoundUp(7.472) = 7.5, which is consistent with the official FIRST calculator and falls within the High category. The resulting vector is CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N (Fig. 4).

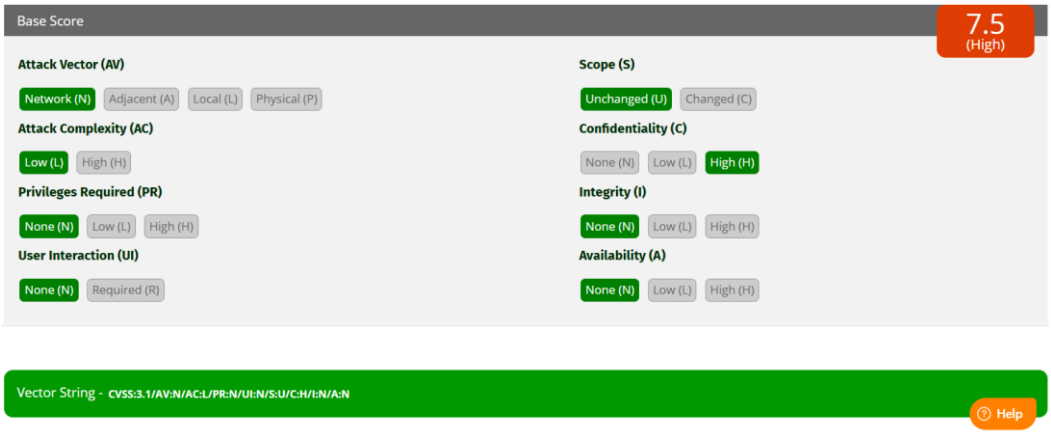


Fig. 4 CVSS v3.1 Base Score of 7.5 (High) for the confirmed SQL injection vulnerability.

V. DISCUSSION

The High rating is driven primarily by the High confidentiality impact: the vulnerability allowed extraction of all user credentials without any authentication. This is aggravated by the finding that passwords were stored in plaintext. Although CVSS v3.1 has no parameter that directly captures credential-storage weakness, the plaintext condition materially worsens the real-world impact, since extracted passwords can be used immediately and enable follow-on attacks such as credential stuffing if users reuse passwords across services. This underscores that CVSS results should be interpreted alongside a contextual understanding of the specific system tested.

The Base Score of 7.5 (High) is consistent with prior SQL injection findings. Utama et al. reported 7.4 (High) and Febrian et al. reported 9.0 (Critical); the score obtained here lies within this range, and numeric differences are expected given variations in application context, system configuration, and exploitation conditions. The Confidentiality = High assignment matches the CVSS v3.1 definition of total loss of confidentiality, satisfied here by the complete extraction of all 15 credential records including plaintext passwords. More broadly, the High category is consistently correlated with serious data-security impact across vulnerability types in the literature, from SQL injection and IDOR to Directory Listing, all converging on unauthorized access to or leakage of sensitive data. The validation therefore rests on the consistency between measured severity and empirically demonstrated impact rather than on numeric identity of scores.

Based on the findings, several mitigations are recommended. First, prepared statements and parameterized queries should replace dynamic query construction so that the database engine strictly separates query structure from user data, preventing characters such as the single quote from altering query logic. Second, server-side input validation and sanitization should verify data type, length, and format, since client-side validation can be trivially bypassed. Third, passwords must be stored using strong, one-way hashing algorithms such as bcrypt or Argon2 with a salt rather than in plaintext. Fourth, the principle of least privilege should be applied to the database account to limit the reach of any successful injection. Fifth, database error messages should be disabled in production and replaced with generic messages, with detailed errors logged server-side. Finally, a Web Application Firewall may serve as an additional defensive layer to detect and block automated injection patterns, although it does not replace source-code remediation.

This study has limitations. It was conducted on a single intentionally vulnerable application using a black-box approach, so root causes at the source-code level were not examined, and destructive operations were deliberately avoided to preserve the laboratory environment, which constrains the assessment of the integrity impact. These constraints define the boundaries within which the reported severity should be interpreted.

VI. CONCLUSIONS

Using a penetration testing approach with a black-box method, this study identified two SQL injection vulnerabilities in the tested web application: an authentication bypass on the login feature, demonstrated with the payload ' OR 1=1 --, and a data-leakage vulnerability on the search feature, confirmed with SQLMap through time-based blind and UNION query techniques and exploited via database enumeration to extract 15 user credential records, including passwords stored in plaintext.

The severity analysis using CVSS v3.1 produced the vector CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N with a Base Score of 7.5 (High), indicating a vulnerability that is remotely exploitable over the network without authentication, whose principal impact is the total loss of data confidentiality. This result was validated against prior work and proved consistent, with the High category reliably correlating with tangible leakage of sensitive user data. The main contribution is an explicit, per-parameter CVSS justification linked to empirical exploitation evidence, addressing the gap left by studies that report only a final score. Future work is encouraged to compare CVSS v3.1 with other risk frameworks (e.g., CVSS v4.0, DREAD, or the OWASP Risk Rating Methodology), to apply gray-box or white-box approaches for deeper root-cause analysis, and to develop a systematic framework for deriving CVSS parameters from penetration testing evidence to reduce assessor subjectivity.

Author Contributions: Reinaldy Thendean: Conceptualization, Methodology, Software, Investigation, Data Curation, Writing - Original Draft. Fenina Adline Twince Tobing: Conceptualization, Methodology, Writing - Review & Editing, Supervision. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Acknowledgments: The authors thank Universitas Multimedia Nusantara for its support, and the certified cybersecurity practitioner who assisted in reviewing the CVSS assessment.

Conflicts of Interest: The authors declare no conflict of interest.

Data Availability: The vulnerable application used in this study is publicly available as the Docker Hub image zxxsnxx/vulnlabvuzlar. No personal data of real individuals were used; all credential records were synthetic data contained in the deliberately vulnerable laboratory application.

Informed Consent: There were no human subjects.

Institutional Review Board Statement: Not applicable.

Animal Subjects: There were no animal subjects.

ORCID

Reinaldy Thendean: https://orcid.org/0009-0003-0989-8144

Fenina Adline Twince Tobing: https://orcid.org/0009-0004-2357-1970

REFERENCES

[1] R. M. Dinata, M. Alzril, M. I. Yamin, H. Effendi, and M. Febriansyah, "Analisis keamanan situs web rumah sakit menggunakan metode penetration testing OWASP," SAINSTECH J. Penelit. dan Pengkaj. Sains dan Teknol., vol. 35, pp. 99-100, 2025.
[2] S. Margareth et al., "Uji penetration testing web server XYZ menggunakan metode OWASP Top 10 dan CVSS," CENTIVE, vol. 4, pp. 1173-1183, 2024.
[3] S. H. Ismail, A. G. Jaafar, and F. A. Rahim, "A review of penetration testing process for SQL injection attack," Open Int. J. Informatics (OIJI).
[4] M. K. Adrian, P. D. Ibnugraha, and H. H. Nuha, "Integrating CVSS, OWASP, and APPI for a comprehensive risk analysis of SQL injection vulnerabilities in e-commerce," Inf. Technol. J., vol. 7, 2025.
[5] S. Neupane, "Detecting and mitigating SQL injection vulnerabilities in web applications," arXiv preprint arXiv:2506.17245, 2025.
[6] OWASP Foundation, "OWASP Top 10: 2025 - The ten most critical web application security risks," 2025. [Online]. Available: https://owasp.org/Top10/2025/A05_2025-Injection/
[7] A. E. Evwiekpaefe and I. Habila, "Implementing SQL injection vulnerability assessment of an e-commerce web application using Vega and Nikto tools," pp. 1-8, 2021.
[8] FIRST.Org, Inc., "Common Vulnerability Scoring System version 3.1: Specification document," 2019. [Online]. Available: https://www.first.org/cvss/v3.1/specification-document
[9] O. Ojagbule and H. Wimmer, "Vulnerability analysis of content management systems to SQL injection using SQLMAP," in SoutheastCon 2018, IEEE, 2018.
[10] F. Boateng, "SQL injection techniques and mitigation strategies: A study using OWASP Mutillidae II," 2025.
[11] "SQL injection attacks prevention system technology: Review," Asian J. Res. Comput. Sci., pp. 13-32, 2021.
[12] R. Vallabhaneni and V. Veeramachaneni, "Understanding penetration testing for evaluating vulnerabilities and enhancing cyber security," Eng. Technol. J., vol. 9, 2024.
[13] V. M. Kozel, I. E. A. Drozdova, O. I. Ivanchuk, and O. O. Prykhodko, "Research of penetration testing methods," vol. 3, pp. 221-227, 2024.
[14] D. Kongara and S. Krishnama, "A process of penetration testing using various tools," Mesopotamian J. CyberSecurity, vol. 2023, pp. 93-103, 2023.

[15] M. R. Fahlevi, D. C. Medika, and F. I. Komputer, "Analisis keamanan aplikasi web menggunakan penetration testing berdasarkan standar OWASP pada DVWA," vol. 21, 2026.
[16] R. S. Wiandani, M. Tahir, I. A. Dyransyha, and R. Ummah, "Identifikasi serangan SQL injection berbantuan aplikasi pengujian keamanan web DVWA," Digit. Transform. Technol., vol. 5, pp. 375-382, 2025.
[17] T. R. Saputra, H. Wardana, A. N. Fariq, R. Cahyono, and Susanto, "Web application security testing against SQL injection attacks using SQL map," J. E-Komtek, vol. 9, pp. 434-439, 2026.
[18] A. Riyanti, B. M. Rahmanto, D. R. Hardianto, R. D. A. Yuristiawan, and A. Setiawan, "Uji penetrasi injeksi SQL terhadap celah keamanan database website menggunakan SQLMap," J. Internet Softw. Eng., vol. 1, p. 9, 2024.
[19] M. F. Begum, L. S. C, and M. P, "Enhancement of web application security using SQLMap and machine learning," Int. Res. J. Innov. Eng. Technol., vol. 9, pp. 267-272, 2025.
[20] "Website penetration testing with SQL injection technique using SQLMap on Termux," J. E-Komtek, vol. 8, pp. 286-293, 2024.
[21] H. M. Harb, D. Eleyan, and A. Eleyan, "SQL injection detection tools advantages and drawbacks," Int. J. Wirel. Microw. Technol., vol. 11, pp. 16-21, 2021.
[22] F. P. Utama, R. Muhamad, and H. Nurhadi, "Uncovering the risk of academic information system vulnerability through PTES and OWASP method," 2024.
[23] D. W. Febrian, R. B. Huwae, and A. Z. Mardiansyah, "Analisis keamanan website perguruan tinggi di Nusa Tenggara Barat terhadap serangan SQL injection, cross site scripting, dan insecure direct object reference melalui pengujian penetrasi," J. Bumigora Inf. Technol. (BITe), vol. 7, pp. 25-38.
[24] D. Rohmaniah, W. M. Ashari, and A. D. Putra, "Enhancing website security using vulnerability assessment and penetration testing (VAPT) based on OWASP Top Ten," p. 404, 2025.
[25] A. A. Permana and E. Sadiah, "Perancangan Sistem Informasi Pembayaran SPP di SMK Tangerang Global," JIKA (Jurnal Informatika), vol. 6, no. 2, p. 113, 2022.
[26] R. Taufiq, A. A. Permana, and M. A. Marfino, "Rancang Bangun Sistem Informasi Pengajuan Cuti Berbasis Web pada PT. Tribuana Gasindo," JIKA (Jurnal Informatika), vol. 6, no. 1, pp. 99-104, 2022.
[27] A. Alanda, D. Satria, M. I. Ardhana, A. A. Dahlan, and H. A. Mooduto, "Web Application Penetration Testing Using SQL Injection Attack," JOIV: Int. J. Informatics Vis., vol. 5, no. 3, pp. 320-326, 2021.
[28] M. Alenezi, M. Nadeem, and R. Asif, "SQL injection attacks countermeasures assessments," Indonesian J. Electr. Eng. Comput. Sci., vol. 21, no. 2, pp. 1121-1131, 2020.
[29] H. Sofyan, M. Sugiarto, and B. M. Akbar, "Implementation of penetration testing on websites to improve security of information assets UPN 'Veteran' Yogyakarta," Telematika, vol. 20, no. 2, pp. 1-10, 2023.
[30] S. Nagpure and S. Kurkure, "Vulnerability assessment and penetration testing of web application," in 2017 Int. Conf. Computing, Communication, Control and Automation (ICCUBEA), 2017, pp. 1-6.